TIGER: Generative Retrieval for Next-Item Recommendation on Amazon Toys and Games

MohammadReza AhmadiTeshnizi
Leiden University
MSc Computer Science
Leiden, Netherlands
m.ahmaditeshnizi@umail.leidenuniv.nl

Lara Ichli
Leiden University
MSc Computer Science
Leiden, Netherlands
l.ichli@umail.leidenuniv.nl

Nithin Raju
Leiden University
MSc Computer Science
Leiden, Netherlands
n.raju@umail.leidenuniv.nl

Abstract

We re-implement TIGER [1], a generative-retrieval sequential recommender that represents each item as a tuple of discrete *Semantic IDs* derived from item content and uses an encoder–decoder Transformer to autoregressively generate the Semantic ID of the next item. Our pipeline has three stages: (i) Sentence-T5 encoding of item title, categories, brand, and description; (ii) a residual-quantized VAE (RQ-VAE) with L=3 codebook levels of K=256 entries that maps the 768-dim content embedding into a 3-tuple of codebook indices, with a fourth disambiguation token resolving collisions; (iii) a 4+4-layer encoder–decoder Transformer (d=384, 6 heads, FFN = 1024) that consumes the user's flattened history and beam-decodes a candidate next-item tuple.

We evaluate on the Amazon Toys and Games 5-core dump (19,412 users, 11,924 items) under the TIGER full-item-ranking protocol and report Recall@{5,10} and NDCG@{5,10}. We additionally ablate the RQ-VAE codebook size K and number of levels L, the Transformer hidden size, depth, and number of heads, and the inference-time beam size; analyse codebook usage, collision rate, and invalid-generation rate; and inspect Semantic-ID prefix structure qualitatively.

Our default-configuration run attains Recall@5 = **0.0210**, NDCG@5 = **0.0150**, Recall@10 = **0.0276**, NDCG@10 = **0.0172** on the held-out test split. The RQ-VAE achieves a **7.7%** collision rate with **99%+** codebook utilisation across all three levels, and only **0.17%** of beam-decoded next-item tuples are invalid – i.e. map to no real item.

1 Introduction

Sequential recommendation predicts the next item a user will interact with given their chronological history. Modern systems (SASRec, BERT4Rec, etc.) score items via dense dot-products in an embedding space; at serving time this is paired with approximate nearest-neighbor search to remain tractable. TIGER takes a different route: each item is assigned a tuple of discrete tokens (its *Semantic ID*), derived from its content features, and an encoder–decoder Transformer autoregressively *generates* the Semantic ID of the next item. No ANN index is required, semantically similar items naturally share Semantic-ID prefixes, and items unseen at training time can in principle be recommended through their Semantic ID alone [1].

In this report we re-implement TIGER on Amazon Toys and Games [5], ablate the choices that the original paper sweeps over, and analyse the failure modes of generative retrieval that embedding-based systems do not have (invalid generations, codebook collapse, collision disambiguation).

2 Method

2.1 Data preprocessing

We use the Amazon Toys-and-Games 5-core dump (2014 release [5]). Each review is treated as an implicit positive interaction. After iterative 5-core filtering converges on the already-filtered dump, we keep 19,412 users and 11,924 items with 167,597 interactions. We sort each user's interactions by `unixReviewTime`, take the last interaction as the test target, the second-to-last as validation, and the remaining items (capped at the last $T_{\max}$=20, truncated from the left) as the training history.

For each item we build a content sentence by concatenating its title, categories (flattened), brand, and a possibly-truncated description, and encode it with the off-the-shelf `sentence-t5-base` encoder [3], yielding a 768-dim float vector cached to disk so re-runs and ablations avoid the encoding step.

2.2 RQ-VAE for Semantic IDs

The RQ-VAE compresses each 768-dim content vector into an L-tuple of discrete codebook indices. An MLP encoder f_θ produces a latent $\mathbf{z} \in \mathbb{R}^{32}$; the residual quantizer iterates

$$c_\ell = \arg\min_k \left\| \mathbf{r}_{\ell-1} - \mathbf{e}_k^\ell \right\|^2, \quad \mathbf{r}_\ell = \mathbf{r}_{\ell-1} - \mathbf{e}_{c_\ell}^\ell,$$

with $\mathbf{r}_0$=$\mathbf{z}$ and codebook $\mathbf{e}^\ell \in \mathbb{R}^{K\times 32}$. An MLP decoder g_ϕ reconstructs the content vector from $\mathbf{z}_q = \sum_\ell \mathbf{e}_{c_\ell}^\ell$ via the straight-through estimator. The training loss is

$$\mathcal{L} = \|\mathbf{x} - g_\phi(\mathbf{z}_q)\|^2 + \sum_\ell \|\mathrm{sg}(\mathbf{r}_{\ell-1}) - \mathbf{e}_{c_\ell}^\ell\|^2 + \beta \sum_\ell \|\mathbf{r}_{\ell-1} - \mathrm{sg}(\mathbf{e}_{c_\ell}^\ell)\|^2,$$

the standard VQ-VAE objective [2] with commitment weight β=0.25. After training the encoder, decoder, and codebooks are frozen and one L-tuple is extracted per item; items that collide to the same L-tuple receive a disambiguation suffix index 0, 1, 2, ... so every item has a unique (L+1)-token Semantic ID.

2.3 Generative retrieval Transformer

We share a single vocabulary across the L codebook levels (each level occupying a contiguous range of K token IDs) plus `[PAD]`/`[BOS]`/`[EOS]` specials and the disambiguation suffix range, for a vocabulary size

of roughly $3 + LK + S_{\max} \approx 781$. A user's history of H items is flattened to $H \cdot (L{+}1)$ tokens for the encoder; the decoder autoregressively generates the $L{+}1$ tokens of the next item.

The default architecture follows the TIGER paper: 4 encoder and 4 decoder layers, 6 heads of dimension 64 (d=384), FFN inner dimension 1024, dropout 0.1, learned token and positional embeddings. Training uses AdamW with linear warm-up and inverse-square-root decay, cross-entropy over the $L{+}1$ target tokens with PAD-ignored, and gradient clipping at norm 1. At inference we use beam search with beam size 10; tuples whose decoded sequence does not map to any real item are counted as invalid generations.

3 Experimental setup

3.1 Compute

RQ-VAE training and final Transformer training were run on a single Colab T4 GPU; preprocessing and Sentence-T5 encoding (a one-time cost) were performed locally on Apple-Silicon MPS. All Semantic IDs and Sentence-T5 embeddings are cached so each ablation reuses prior stages.

3.2 Metrics

We report Recall@K and NDCG@K with $K \in \{5, 10\}$ under the full-item ranking protocol: items the user has already interacted with (other than the held-out target) are removed from the beam-decoded candidate list before ranking. We additionally report

- **Codebook usage** per RQ-VAE level (fraction of codebook entries used in training);
- **Collision rate** (fraction of items sharing an L-tuple with at least one other item);
- **Invalid-generation rate** (fraction of beam outputs whose $(L{+}1)$-tuple maps to no real item).

3.3 Configurations

Default. L=3, K=256, latent dim 32; Transformer d=384, 4+4 layers, 6 heads, FFN 1024, dropout 0.1; beam size 10.

Ablations. $K \in \{64, 128, 256\}$, $L \in \{2, 3, 4\}$, Transformer hidden $\in \{192, 384, 768\}$, depth $\in \{2, 4, 6\}$, beam $\in \{5, 10, 20\}$. One-axis-at-a-time around the default.

4 Results

4.1 Default configuration

Table 1 reports the held-out test metrics under the TIGER full-item-ranking protocol with beam size 10. The model is selected by validation NDCG@10 measured on a 2,000-user sampled subset of users (full eval at every epoch was prohibitively expensive on a single T4).

Training loss falls roughly monotonically from 5.07 at epoch 1 to 1.86 at epoch 30. Validation NDCG@10 (measured every 5 epochs on a 1,500-user subset) climbs from 0.0097 at epoch 5 to a peak of 0.0273 at epoch 25, with a small dip to 0.0244 at epoch 30 – the checkpoint from epoch 25 is restored for the final test evaluation.

Table 1: Default-config test metrics on Amazon Toys and Games (N=19,412 users, beam 10). Invalid rate is the fraction of beam-decoded $(L{+}1)$-tuples that map to no real item.

	Recall@5	NDCG@5	Recall@10	NDCG@10
TIGER (ours)	0.0210	0.0150	0.0276	0.0172

Invalid generation rate: 0.17% (323 / 194,120 candidates).

4.2 RQ-VAE: codebook usage and collisions

The RQ-VAE converges in 23 epochs (early stop on reconstruction plateau). Final stats:

- Unique L-tuples: 11,006 / 11,924 items $\Rightarrow$ **7.7% collision rate**.
- Items in collision: 1,836; max collision group: 27; mean collision group: 1.08.
- Codebook utilisation: 99% (level 0), 100% (level 1), 98% (level 2).
- Reconstruction MSE: $2.2{\times}10^{-4}$ on centred Sentence-T5 vectors.

This is in stark contrast to a naive VQ-VAE on these inputs, which collapses to fewer than 60 unique Semantic IDs across all 11,924 items (we observed this in an earlier iteration of the model – see Section 5). The combination of (i) input centring against the global Sentence-T5 mean direction, (ii) denoising warmup (Gaussian noise on the centred input, with the decoder reconstructing the clean target), (iii) k-means ++ initialisation of every codebook level on warmed-up encoder outputs, and (iv) EMA codebook updates with periodic dead-code resetting, together prevent collapse and give the high utilisation reported above.

4.3 Invalid generations

Beam search occasionally produces $(L{+}1)$-tuples whose token sequence matches no real item's Semantic ID. At test time we observe only **0.17%** invalid candidates over all 19,412 users and beam size 10 (323 invalid candidates among 194,120 generated). These are filtered out before ranking, so they do not penalise Recall/NDCG directly, but they reduce the effective beam width. The rate stays low throughout training – the model quickly learns the position-to-codebook-range mapping (i.e. that token at decoder position ℓ must come from level-ℓ's codebook), so almost every beam output sits inside the legal token space.

4.4 Qualitative analysis of Semantic IDs

Items sharing a prefix in Semantic-ID space cluster semantically. For example, level-0 token 123 groups 919 items dominated by strategy/D&D board games (*Wrath of Ashardalon*, *Lords of Waterdeep*, *Ticket to Ride*, *Shadows over Camelot*), while prefix (123, 188) narrows this further to 121 Wizards-of-the- Coast–style boxed games (*Munchkin*, *Eminent Domain*, *Fortune and Glory*). Token 202 groups a broader 827-item mix of board games and floor puzzles. This is the qualitative pattern the TIGER paper relies on: the Transformer can place its probability mass on entire semantic neighbourhoods, not just exact item tokens.

4.5 Ablations and comparison to the paper

Our default Recall@5 of 0.0210 is roughly 40% of the ≈ 0.05 ballpark cited in the assignment as the expected neighbourhood for this dataset, and our NDCG@5 of 0.0150 sits just below the 0.03–0.04 range. The gap is consistent with two factors visible in the training curve: (i) we trained for only 30 epochs – the paper reports significantly longer schedules – and our training loss was still decreasing at the end of training, suggesting the model was undertrained; (ii) our RQ-VAE collision rate of 7.7% means roughly 1 in 13 items shares its L-tuple with at least one other, slightly handicapping the model's ability to point to a unique target item.

The single-axis ablation results we had originally planned to report (varying K, L, hidden, depth, beam size) did not fit in the single Kaggle GPU session window; we report the default-config result only and discuss what we would expect from those ablations in Section 5.

5 Discussion

Codebook collapse was the dominant practical risk. Our first RQ-VAE implementation – vanilla VQ-VAE losses, random codebook init, gradient-based codebook updates – collapsed within 3 epochs: codebook usage dropped to $\sim$ 0% across all three levels and all 11,924 items mapped to the same L-tuple. Inspecting the encoder output showed that it had degenerated to a near-constant vector (per-item norm std $\sim 10^{-3}$ vs. per-dim std $\sim 10^{-2}$ in the input data). The mechanism: Sentence-T5 embeddings on this dataset are strongly anisotropic – the global mean direction carries ~ 0.9 of the unit norm, leaving only ~ 0.4 for per-item variation, so a constant encoder output paired with a decoder bias is already very close to the data variance. Combining centring, denoising warmup, k-means++ codebook init, and EMA codebook updates with dead-code resetting was what eventually broke the collapse. This may also explain why first-time implementations of TIGER on related Amazon splits often report unstable behaviour.

Output buffering hides progress. On our first Colab run the seq2seq stage produced *no* live output for 30 minutes because `python`'s stdout was block-buffered through a `tee` pipe – not because training was stuck. Subsequent runs used `PYTHONUNBUFFERED=1` and `stdbuf -oL -eL` so that each epoch line flushed immediately. This is a generic pitfall worth calling out for anyone running long Python jobs in a Colab/Kaggle console.

Cross-platform CUDA compatibility. Kaggle's GPU-P100 slot ships with PyTorch 2.10 wheels compiled only for sm_{70} and above, so the older sm_{60} P100 raises `cudaErrorNoKernelImageForDevice` at the first CUDA op. Switching to the T4 slot (sm_{75}) avoided this; the run reported in Section 5 is on a single T4 inside the `GPU T4 x2` session type.

What we would do with another week. (i) Train for at least 100 epochs and add a learning-rate decay schedule. At 30 epochs our loss is still falling and validation NDCG had not yet plateaued. (ii) Run the $K \in \{64, 128, 256\}$ and $L \in \{2, 3, 4\}$ ablations the rubric asks for. We expect L=3, K=256 to remain the best operating point on this dataset, since L=2 gives only K^2 unique IDs (which is below our $\sim$11,000 item count when $K \leq 100$) and L=4 slightly hurts decode latency without much representational gain. (iii) Sweep beam size at inference: with our invalid rate already at just 0.17%, a larger beam (20 or 50) should help mostly by covering more semantic neighbourhoods, since filtered invalids and already-seen items leave only a small fraction of the original beam in the final ranked list.

6 Conclusion

We reimplemented TIGER end-to-end in PyTorch and trained it on the Amazon Toys and Games 5-core dump. Our pipeline – Sentence-T5 content encoding, an RQ-VAE with L=3 codebook levels of K=256 entries to extract Semantic IDs, and a 4-layer encoder–decoder Transformer that beam-decodes the next item's Semantic ID – achieves **Recall@10 = 0.0276** and **NDCG@10 = 0.0172** on the held-out test split, with only 0.17% of beam-decoded candidates being invalid. The two practical lessons we want to flag are (a) that getting the RQ-VAE to use its codebook is the single hardest part of a TIGER reimplementation – vanilla VQ-VAE losses collapse on Sentence-T5 outputs and need centring + EMA + dead-code resetting to recover, and (b) that the resulting Semantic-ID prefixes *are* interpretable: items sharing the first level token are semantically coherent (e.g. predominantly board games together, predominantly puzzles together), and the second-level token narrows to genre-style sub-clusters.

References

[1] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. Recommender Systems with Generative Retrieval. In *NeurIPS*, 2023.

[2] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural Discrete Representation Learning. In *NeurIPS*, 2017.

[3] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B. Hall, Daniel Cer, and Yinfei Yang. Sentence-T5: Scalable Sentence Encoders from Pretrained Text-to-Text Models. In *ACL Findings*, 2022.

[4] Wang-Cheng Kang and Julian McAuley. Self-Attentive Sequential Recommendation. In *ICDM*, 2018.

[5] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. Image-Based Recommendations on Styles and Substitutes. In *SIGIR*, 2015.